

Pre-cleaning story: how we got to know our 11 tables

Oct 4, 2026 · @Daniel

Where we started

We were confused about where to start, so we decided to understand our tables before cleaning or engineering anything. The dataset has 14 tables, the brief asks for exploratory analysis before and after cleaning, and it warns that results, points and standings are "future" information. Our share is 11 tables: `seasons`, `circuits`, `drivers`, `constructors`, `status`, `sprint_results`, `driver_standings`, `constructor_standings`, `constructor_results`, `lap_times` and `pit_stops`.

We set two rules before running anything. First, we look but we do not change: the raw files stay untouched and every step works on copies. Second, we do not conclude anything unless we have seen it in something we ran, because the brief wants every choice justified with evidence and not by looking at rows by eye.

For every table we ask the same questions:

- What is one row?
- Which column identifies it?
- What is missing, and why?
- When is it written, and could we know it before the race?
- How far back does it go?
- Which values look wrong?

Answering them in this order gave the notebook its structure: setup (Part 0), reference tables (Part 1), race-level tables (Part 2), lap-level tables (Part 3), all tables together (Part 4) and what we learnt (Part 5). The step numbers in this story match the notebook, and every number is one the notebook prints.

Part 0 · Setup: loading everything untouched

Step 0.1 · Loading the tables and mapping them

The first thing we did was load all 11 tables exactly as delivered, every column as text. The dataset writes a missing value as the text `\N`, and we wanted to count those instead of letting pandas hide them. We also saved a fingerprint (a hash) of every raw table, so that at the end we can prove nothing changed.

Then we wrote a map of the 11 tables: what one row is, which key must be unique, how many rows and columns, and where `\N` appears. We saw that `lap_times` is the giant (589,081 rows), that `seasons` and `status` are tiny lists, and that `\N` appears in only 3 tables and 8 columns. So we knew missing values are rare, and that we can explain each of those 8 columns one by one. Later we collect them in a single table (step 4.3).

Step 0.2 · Is `results` really one row per driver per race?

Before looking at our own tables we stopped at a table that is not ours. We count race starts and compare points against `results` all the time, and the brief says the modelling table needs exactly one row for each (`raceId` , `driverId`). So we tested that pair.

We saw 26,759 rows but only 26,668 different pairs: 85 driver-races appear more than once, in 42 races, nearly all in the 1950s. In the 1951 French GP, Fangio and Fagioli each have two rows: one for the car they started (11th, 0 points) and one for a car that won. Even with one row per driver, 18 races between 1950 and 1960 have more than 3 drivers with `positionOrder <= 3`.

So these are the shared-drive duplicates the brief warns about. We did not choose a rule for them: which row to keep, and how to label those 18 races, is for the cleaning task, and we will have to justify it there. We only built a counting version (points added up, best finish kept), so that our own numbers are right.

Part 1 · The reference tables

We started with the five lists that every other table points at: `seasons` , `circuits` , `drivers` , `constructors` and `status` . If they are wrong or ambiguous, every join built on them is wrong.

Step 1.1 · Seasons and circuits (prepares Question 1)

We started with the two smallest lists. For `seasons` we checked that every year from 1950 to 2024 is there. For `circuits` we asked a question that comes from Question 1: it compares circuits before and after 2014, so how many races did each circuit host in each era?

We saw 75 valid seasons with 7 to 24 races each, and 77 circuits with valid coordinates. 11 circuits hosted a single race. 69 circuits were used up to 2013 but only 32 from 2014, and only 24 are in both eras. Asking for at least 5 races in each era leaves 16 circuits; at least 10 leaves 8. We also found two traps: in 2020 and 2021 one circuit hosted two races in the same season, and 12 circuits hosted races under more than one name.

So we knew a circuit ranking can only use a small pool, and that a circuit must be identified by `circuitId`, never by a race name. Later, Question 1 needs a minimum number of races per circuit in each era. We lean towards 5 per era (16 circuits), because 10 would leave only 8. Seasons of 7 to 24 races also tell us that counts per season must become rates.

Step 1.2 · Drivers: which column identifies a driver?

Next came `drivers`. Everything joins to it, so we asked which column can safely identify a driver, and why `number` and `code` are mostly `\N`.

We tested every candidate. Only `driverId`, `driverRef` and `url` are unique and complete. `code` is shared by 7 pairs of drivers (ALB is both Albers and Albon), 11 car numbers are reused and 45 surnames are shared. Then we asked whether the gaps are random, and grouped the drivers by the decade of their first race. Every driver who debuted from the 1950s to the 1980s has neither number nor code. For 2000s debuts, 78% have no number and 27% no code. For debuts from 2014, nothing is missing.

So the gaps follow the era: the values did not exist yet for older drivers. Later we keep them missing, we never fill them, and we do not use them as model inputs, because they only tell us the era. We join on `driverId` only.

Step 1.3 · Drivers: dates of birth and age

"Age on the race day" is a feature we know before the race, so we checked that the dates of birth can support it. This was also our first typing check: text to real dates.

All 861 dates converted (the code would stop on a single failure), none is missing, and only 7 drivers were born on 1 January, where 2 or 3 would be expected by chance. So there is no placeholder habit. We then computed the age at every race: from 17.5 (Verstappen, 2015) to 58.8 (Chiron, 1958), with no negative age and nobody born after their first race. The median age of the grid fell from 39.0 in 1950 to 30.3 in 1980 and 27.9 in 2024.

So `dob` becomes a date and `driver_age` is a feature we may use. The falling age tells us that age behaves differently in different eras, which we will mention as an era-sensitivity limitation in the report.

Step 1.4 · Countries and nationalities (prepares Question 2)

Question 2 asks whether drivers podium more often at home. A home race needs a driver's nationality to match the circuit's country, so we asked how both sides are written.

We saw 35 spellings of circuit countries, three of them abbreviations, and the USA written twice (`USA` and `United States`). None of the 35 is spelled like a driver nationality: circuits say "UK" and drivers say "British". A direct join would find no home race at all. So we wrote a map by hand. 25 nationalities map to a circuit country, 13 have no circuit in their country, and 4 are mixed or historical (American-Italian, Argentine-Italian, East German, Rhodesian). With the map, 2,485 of the 26,668 race entries (9.3%) are home races.

Then we asked whether some races distort this. The Indianapolis 500 counted as a championship race from 1950 to 1960, and 380 of its 381 entries are home entries. Without it, the American home entries fall from 500 to 120. So we knew it would make home advantage look far bigger for Americans. Later, Question 2 decides between excluding it and reporting with and without it. We lean towards excluding it, because almost every entrant was American. The four mixed nationalities are decided there too.

Step 1.5 · Constructors: is one `constructorId` one continuous team? (prepares Question 3)

Question 3 compares constructors, and team form will be a feature. Both assume a constructor is one continuous team, so we tested that.

We saw 212 constructors, 70 of which raced a single season. One real team is often split over several ids: 50 constructors share a Wikipedia page with another one (Cooper has 11 ids, Lotus 7, Brabham 6). The opposite also happens: one id covers separate periods, for

example Mercedes with a break of 54 seasons, Alfa Romeo with 33 and Aston Martin with 60. Question 3's periods have 19 constructors (2014 to 2021) and 12 (2022 to 2024), and 31 constructors only ever raced the Indianapolis 500.

So a `constructorId` is a valid key, but not a team with a continuous history. Later we build team form from a recent window only (for example the previous one or two seasons), so that a gap resets it. For Question 3 we set a minimum number of starts and mention the renames. We lean towards leaving out the Indianapolis 500 chassis, because they are builders, not teams.

Step 1.6 · Status: the 139 reasons a race ended (prepares Question 3)

Question 3 needs the mechanical-retirement rate. The brief defines it as a start that ended early because a part of the car failed, excluding crashes, driver errors and regulation decisions. `status` has 139 reasons, so we sorted them into groups as a first draft.

We saw 58 clearly mechanical statuses, 5 accident or collision, and 29 in a grey zone such as `Retired`, `Wheel`, `Tyre` or `Puncture`, where the cause is unclear. With the clear statuses only, 9.2% of race starts in 2014 to 2021 and 5.6% in 2022 to 2024 ended mechanically. Counting the grey zone too, the rates become 11.4% and 7.0%. We also saw that 99.8% of podium finishers have the status `Finished` or `+N Laps`, so the status is basically the outcome.

From this we knew two things. The answer to Question 3 depends on how we treat the grey zone, so later we either report both definitions or argue each grey status one by one. We lean towards reporting both and naming one as the headline, because the data cannot settle the grey statuses by itself. And the status is only known after the race, so it can never be a model feature.

Part 2 · Race-level tables

The next four tables summarise what happened in or after a race: `sprint_results`, `driver_standings`, `constructor_standings` and `constructor_results`. For each of them the question that matters most is *when is it written*, because the brief treats results, points and post-race standings as leakage. We wanted evidence for the answer, not an assumption.

Step 2.1 · sprint_results: what is in it, and is it a useful feature?

The sprint is run on Saturday, before the Sunday race, so in time it is allowed. We wanted to know whether the table is complete enough to be a feature, and whether it adds anything the Sunday grid does not already give us.

First we looked at what is in it. The 360 rows are 18 sprints with exactly 20 drivers each, which is only 1.6% of all races (3, 3, 6 and 6 sprints in 2021 to 2024). The empty cells belong to drivers who did not finish: 15 have no position and 20 have no time. The time text ("25:38.426", "+1.430") matches the numeric `milliseconds` in all 340 rows that have one. The top 3 scored 3-2-1 points in 2021 and the top 8 scored 8 to 1 from 2022.

Then we compared three orderings of the same weekends: sprint finish, qualifying and the Sunday grid. In 2021 and 2022 the sprint order is almost the Sunday grid (rank correlation 0.89 and 0.99), while from 2023 qualifying is (0.99 and 0.96) and the sprint much less (0.62 and 0.54). The sprint's top 3 reached the Sunday podium 33 times out of 54, the grid's top 3 30 times out of 53.

So a sprint feature would be empty for 98% of the races, and in 2021 and 2022 it repeats the grid. Later the team decides. We lean towards leaving it out of the main model, at most as an optional experiment with a `has_sprint` flag, because it adds nothing beyond the grid. We keep in mind that sprint points sit inside the standings and team scores of 2021 onward.

Step 2.2 · driver_standings: what is one row?

`driver_standings` looks like a ready-made feature ("points so far"). Before using it we asked what a row really is, whether every driver has one, and which values look strange.

We saw 34,863 rows with unique keys and standings for all 1,125 races. A row is a running total, not "who raced today": in the 1950s the median race has 71 standings rows for 21 starters, and 8,664 rows belong to a driver who did not start that race. Points and wins never go down inside a season, and `position` runs exactly 1 to n in every race. One row has `positionText` D (Michael Schumacher, 1997, position 26 with 78 points). 276 rows have decimal points, caused by only 22 races. 469 race starts have no standings row, 447 of them in the 1990s and 2000s, but every start that scored points has one (0 of 8,167). Finally, the champion's final points go from 30 (1950) to 95 (2009), 256 (2010) and 575 (2023).

So we knew a missing row can only mean "no points yet". Later such a driver gets 0 points and 0 wins before the race, but an unknown position, never 0. Raw points cannot be compared across seasons, so we plan to use the position, the gap to the leader or the

share of the leader's points instead.

Step 2.3 · driver_standings: is a row written before or after its race?

This is the question that matters most. The row stored under a `raceId` could be the standings before that race or after it, and the table does not say. If it is after, using it to predict that race would hand the model the answer. So we tested it: a row written after its race must add exactly the points the driver scored that weekend, Sunday race plus sprint.

It does, in 34,781 of 34,863 rows (99.8%), and in 100% of the rows from 1991 on. In 2021 to 2024 the Sunday points alone match only 93.2%, Sunday plus sprint match 100%. The 82 rows that disagree are between 1950 and 1990, for example Clark in 1963 (73 points scored, 54 official) and Prost in 1988 (105 scored, 87 official), which fits the old rule that only a driver's best results counted. Next we built the safe version: the driver's row from the previous round of the same season. It equals the points scored in earlier races in 99.75% of starts after round 1. We also saw that `raceId` is not in date order (61 ids are earlier than the id before them), while `round` is.

Then we measured the leak. Ranking drivers by their standings agrees with the finishing order at a rank correlation of 0.46 on average with the standings from before the race, but 0.57 with the standings from the same race. The race winner leads the table after the race in 48.4% of races, but led it before in only 32.8%.

So we knew that for a race only the previous round's standings may be used, shifted by season and round and never by `raceId`. Round 1 has nothing before it, so later we choose a documented start value plus a flag. We lean towards 0 points with an unknown position, because that is what "no race yet" means. The 0.46 against 0.57 is the evidence for the report's leakage section.

Step 2.4 · constructor_standings: the same questions for teams

We asked the same questions of the team championship, expecting the same answers. Most were the same, but not all.

The table starts in 1958: none of the 64 earlier races has team standings. It is a running total and written after the race: the points a row adds equal the team's `constructor_results` points in 99.3% of 12,497 rows, and in 2021 to 2024 they match `constructor_results` (100%) but not `constructor_results` plus sprint points (90.9%). So `constructor_results` already contains the sprint points. We also found differences. Team points can go down, twice (Force India 2018 from 59 to 18, Racing Point 2020 from 42

to 41). `positionText` is `E` in 17 rows, all McLaren 2007. And 129 team starts have no row, 10 of which scored points, all at the Indianapolis 500 of 1958 to 1960. The previous-round version equals the points scored in earlier races in 98.8% of team starts.

So the rule is the same as for drivers (previous round, shifted by season and round), but "every scorer has a row" does not hold for teams. Later we either flag the missing team standings, start the team window in 1958, or take team form from `results`. We lean towards `results`, for the reasons in the next step.

Step 2.5 · `constructor_results`: which source should team form use?

This table gives each team's points per race, so it is a candidate source for team form. A feature must mean the same thing in every season, so we checked whether a team's score always equals the sum of its drivers' points.

It equals it in 12,234 of 12,618 team-races (96.96%), and in 12,316 (97.61%) once sprint points are added. From 1979 on all 9,911 team-races are identical. In 2021 to 2024 all 82 differences are team-races with sprint points. From 1958 to 1978 the team scored less than its drivers in all 302 remaining differences: 292 equal the best car only. We also saw that `status` is `D` only for McLaren 2007, the same 17 rows as the `E` rows above, and that the exclusion is not in the points. 7 rows have no team start in `results`, and 1958 has no rows for the Indianapolis 500.

So `constructor_results` changes meaning over time (best car only until 1978, sprint points from 2021), while in `results` every car counts the same way in every season. Later we build team form from `results`, from earlier races only, and we drop `status`. For 2014 to 2024 both sources agree apart from sprint weekends.

Part 3 · Lap-level tables

`lap_times` and `pit_stops` describe what happens inside a race, so for the race we predict they are future information. We already knew they can only be used from earlier races. The questions here were different: which races have the data, which values are extreme, and does it agree with `results`?

Step 3.1 · `lap_times`: what is one row, and which races have it?

Lap times could tell us how fast a driver or a team has been recently. Before looking for odd laps we needed to know what a row means, where lap data exists, and whether the time text can be trusted. This was our second typing check, lap-time text to numbers.

We saw 589,081 rows with a unique key, and every driver-race exists in `results`. Lap data exists only from 1996: all 544 races of 1996 to 2024 and none before, which is 48.4% of all races. A row is one lap of one driver: every driver-race has laps 1 to n with no gap, and `position` is the running order after that lap, not the finishing position. The shortest race is the 2021 Belgian GP with a single lap, and the smallest field is the 2005 US GP with 6 cars. The `time` text equals `milliseconds` in all 589,081 rows. 589,038 are written `m:ss.mmm`, but 43 are written `h:mm:ss.mmm`, in the 2011 Canadian GP (about 2 hours) and the 2014 British GP (about 1 hour). Those are red-flag stoppages, not real laps.

So we use `milliseconds`, and if the text is ever parsed the parser must handle hours. Lap-based features exist only from 1996, so they need a flag or a start in 1996. From 2014 on no race is missing. And the laps of a race may only be used for later races.

Step 3.2 · `lap_times`: extreme laps, and does it agree with results?

A lap time is only a useful pace signal if it is a real racing lap and if circuits can be compared. We also wanted to know how far to trust `lap_times` against the official `results`.

The fastest lap is 55.4 s and the slowest 125.1 minutes (the 2011 red-flag stop). The typical lap of a race goes from 58.7 s to 225.4 s, so raw lap times cannot be compared across races. Divided by the typical lap of its own race, no lap is impossibly fast. 6.6% of the laps are slower than 1.2 times the typical lap and 1.99% (11,709) slower than 1.5 times. For the 6,473 laps slower than 1.5 times since 2011, when pit data exists, 493 are lap 1, 903 are a pit-stop lap, 1,222 follow a stop, and 3,855 (60%) have other causes such as safety cars, red flags, spins or rain. The lap counts agree with `results` in 10,951 of 11,041 driver-races (99.18%). In the 2014 Chinese GP all 20 classified cars have 2 more laps in `lap_times` than in `results`. The fastest lap of a driver is the same in both tables in 97.5% of 8,249 driver-races.

So we never use raw lap times: each lap is divided by the typical lap of its race. For a pace feature we keep only clean racing laps, which means dropping lap 1, pit and out laps, and laps above a cutoff. Later the team chooses the cutoff. We lean towards starting at 1.2 times (it removes 6.6% of laps) and testing 1.5 times (1.99%), because the 1.5 cutoff would still keep slow safety-car laps. The 90 lap-count differences are recorded, not fixed; if a feature needs lap counts it uses `results.laps`.

Step 3.3 · pit_stops: what is one row, and how complete is it?

Pit stops tell us about a race's strategy and a team's pit-crew speed. Before looking for odd stops we asked what a row means, where the data exists, whether it is complete, and how the two time columns are written.

We saw 11,371 rows with a unique key. The data exists only from 2011: 285 races, 25.3% of all races, and every race of 2011 to 2024 except the 2021 Belgian GP (one lap, so no stops). It is complete for finishers: of 4,922 classified finishers only 3 have no stop, while 656 of the 1,038 non-finishers have one, so "no stop row" for an early retirement is normal. The average number of stops per finisher moves from 2.68 (2011) to 1.42 (2018) and 2.20 (2023). `duration` is text but equals `milliseconds` in all 11,371 rows, with 517 rows written with minutes (for example `16:44.718`). `time` is the local clock time of the stop: the 2011 Australian GP starts at 06:00:00 in `races` but its first stop is at 17:05:23.

So we use `milliseconds` for the duration and drop `time`, because no feature needs it and it cannot be combined with `races.time`. Pit-based features exist only from 2011, so they need a flag. Stop counts are not comparable across seasons, and the stops of a race may only be used for later races.

Step 3.4 · pit_stops: real stops versus red-flag pauses

Step 3.3 showed stops of up to 51 minutes and a stop number up to 70. A real pit stop takes about 24 seconds, so a stop time or a stop count only means something once real stops are separated from red-flag pauses and from data artifacts.

We saw a median of 23.6 s, with 90% of stops under 31.6 s. 517 stops are longer than 60 s, in 50 races: 478 are red-flag pauses (longer than 5 minutes, in 26 races) and 39 are very slow stops. The races with the most are the 2023 Australian GP (49 of its 65 stops), the 2016 Brazilian GP (36 of 62) and the 2021 Saudi Arabian GP (35 of 47). We also found 8 driver-races with odd stop numbers. At the 2024 Monaco GP six drivers have a 39-minute stop on lap 1 and then a normal stop on lap 2 numbered between 15 and 70 instead of 2, so the 70 is a label error and not 70 stops. Removing the pauses changes the stop count in 404 of 5,575 driver-races.

So a real stop is up to 60 s, a stop from 60 s to 5 minutes is very slow but still a stop, and a stop over 5 minutes is a red-flag pause that is removed from durations and counts. Later we use medians of normal stops, and we recount stops by sorting on the lap instead of trusting the `stop` number.

Part 4 · All tables together

Until now each table was looked at on its own. The last four checks look at all of them at once: do they fit together, how far back does each go, what exactly is missing, and are the raw files still intact?

Step 4.1 · Keys and links

The brief requires that primary keys are unique and non-null, and that foreign keys reconcile, meaning every id exists in the table it points to. We tested both for all our tables, and we explained every row that has no partner in `results`.

All 15 keys are valid, with no duplicate and no missing value. All 20 links reconcile with 0 orphans, and the notebook stops with an error if that ever changes. The rows without a partner in `results` are all ones we had already explained: `driver_standings` has 8,664 (the carried-forward drivers of step 2.2), `constructor_standings` has 892 (step 2.4) and `constructor_results` has 7 (step 2.5); `sprint_results`, `lap_times` and `pit_stops` have none. Only 3 reference rows are never used: the constructor Eagle and the statuses `+49 Laps` and `+38 Laps`.

So nothing needs fixing, and every count above zero has a known reason. For the cleaning task this gives a join recipe: build the driver-race table from `results`, left-join the other tables on `(raceId, driverId)` or `(raceId, constructorId)`, and take the standings from the previous round of the same season. The extra rows then drop out by themselves.

Step 4.2 · Coverage over time

Our tables do not all start in the same year, so any feature built from them is missing in the early seasons. The brief lists coverage drift as a limitation to report, so we measured it for every table and every season and drew it (Figure 11).

The coverage steps up four times. `driver_standings` and `results` cover all 1,125 races from 1950. The team tables start in 1958 (94% of races), `lap_times` in 1996 (48.4%), `pit_stops` in 2011 (25.3%) and `sprint_results` in 2021 (1.6%). Only a few seasons are partly covered: `constructor_results` in 1958, `pit_stops` in 2021 and the sprint seasons. For the two periods of the brief, 2014 to 2021 and 2022 to 2024, every table covers 100% of the races, except `pit_stops` in 2014 to 2021 (99.4%) and `sprint_results` (1.9% and 22.1%).

So lap and pit features simply do not exist for 52% and 75% of all races, and the figure goes into the report's limitations. Later we give those features flags (`has_lap_data` , `has_pit_data` , `has_sprint`) and we never fill earlier seasons with an average. If one model is trained on all seasons, we plan a core feature set that exists everywhere and an extended set with laps and pit stops, and we report the score per era. We lean towards this comparison because it shows the drift instead of hiding it.

Step 4.3 · Missing values and type conversions

The brief asks us to clean the placeholder `\N` and to apply semantic typing, such as dates to dates and lap-time strings to numeric seconds. Before cleaning we listed exactly where `\N` is, why it is there, and which conversions are needed, and we tested every conversion on all rows so that the cleaning step cannot lose anything.

8 columns contain `\N` , in 3 of our 11 tables, and each has a reason we found earlier. `drivers.number` (93.15%) and `drivers.code` (87.92%) follow the era. Five `sprint_results` columns (2.5% to 5.6%) are empty for drivers who did not finish. `constructor_results.status` (99.87%) only holds the flag `D` . The other 8 tables have no missing value, and no table has an empty cell. The conversions work on every row: 861 of 861 dates of birth, 589,081 of 589,081 lap times, 11,371 of 11,371 pit durations and 340 of 340 sprint times.

So the cleaning task converts exactly these columns, turns `\N` into a real missing value in the cleaned copies, and fills nothing without a reason. `number` and `code` stay missing, and the empty sprint cells keep meaning "did not finish".

Step 4.4 · The raw files were never changed

The brief says raw files stay immutable and cleaning happens on copies. In step 0.1 we stored a fingerprint of every raw table, so we computed it again and compared. All 13 raw tables (our 11, plus `races` and `results`) have the same fingerprint as when they were loaded, so 0 changed, and the notebook stops with an error otherwise. The raw layer is intact, and the cleaning task starts from these files.

Part 5 · What we learnt and what comes next

Step 5.1 · What this audit changed in our plan

We did not clean anything yet, but we already changed the plan in seven concrete ways. Each one comes from a number we saw, not from a guess.

- **Leakage.** The standings stored under a race already contain that race and its sprint, and they look better than they should (0.57 against 0.46). So features use only the previous round of the same season, ordered by season and round.
- **Grain.** `results` repeats 85 driver-races (shared drives), and 18 races in the 1950s have more than 3 podium drivers. So the cleaning task must choose and justify one rule for the modelling table.
- **Team form.** `constructor_results` changes meaning over time, so team form comes from `results`, from earlier races only.
- **Coverage.** Team tables start in 1958, laps in 1996, pit stops in 2011 and sprints in 2021. So these features get flags and are never filled with an average.
- **Missing values and types.** `\N` sits in only 8 columns, each with a reason, and every date and duration converts without loss. So cleaning is a precise list, not a search.
- **The three questions.** Only 16 circuits have at least 5 races in both eras (Question 1). Countries and nationalities need a written map, and the Indianapolis 500 inflates American home races about four times (Question 2). The mechanical rate depends on a grey zone of 29 statuses (Question 3).
- **Scales.** Seasons have 7 to 24 races, champions score 30 to 575 points, and a lap is only comparable to its own race. So features use rates, shares, positions and relative laps.

The raw files are unchanged, and the notebook stops with an error if a key breaks, a link breaks or an unexplained missing value appears.

Step 5.2 · What is still open

These are choices the data cannot make for us. For each one we give our lean and the step that holds the evidence. The team decides.

#	Open decision	Our lean	Step
1	Use the sprint in the main model?	No, at most an optional experiment with <code>has_sprint</code>	2.1

#	Open decision	Our lean	Step
2	Cutoff for a clean lap	Start at 1.2 times the typical lap, test 1.5 times	3.2
3	Start value of standings features in round 1	0 points, unknown position, plus a flag	2.3, 2.4
4	Team form before 1958 and in round 1	Take it from <code>results</code> for every season	2.4, 2.5
5	Question 3: the grey statuses	Report both definitions and name one as the headline	1.6
6	Question 2: the nationality map	Finish it, decide the 4 mixed nationalities, exclude the Indianapolis 500	1.4
7	Question 1: minimum races per circuit	At least 5 per era (16 circuits)	1.1
8	Question 3: minimum starts per constructor	Set one, mention renames, exclude Indianapolis 500 chassis	1.5
9	Source of a driver's best lap	One source only, not a mix	3.2
10	Seasons a model trains on	Core set against extended set, scored per era	4.2
11	The podium label before 1960	Choose one rule for repeated driver-races and the 18 odd races	0.2

The next stage is the cleaning task. It starts from the raw files, builds the driver-race table from `results`, and applies the rules above to copies.